<u>Algorithms, AI, & ML Lab: Assignment 4</u>

Decision Tree & Random Forest Classifiers

1. **Prepare the dataset in desired representation:** Read the provided dataset. Prepare the data sample matrix <u>*Examples*</u>, decision class as <u>*Target_attribute*</u>, and list of the features as <u>*Attributes*</u> from the dataset. Data curation should be generic enough to be applicable to any other randomly selected dataset provided in the similar format.
2. **Implement the Decision Tree learning ID3 algorithm**: Follow the generic ID3 algorithm as sketched in Table 3.1 (Page 56) of the Text-Book "Machine Learning" by Tom M. Mitchell (McGraw-Hill publications, 1997), and reproduced at the end of this assignment for ready reference. Provide adequate comments describing major steps of the algorithm to make it easy to read. Also, make sure that the developed code is generic enough with below hyper-parameters of the Decision Tree model and ID3 algorithm can be adapted easily as per the need of the application.
    - **a. Metric for deciding the best attribute at every node (*Impurity_Metric*)**: Manually selected choices can be: (i) Entropy-based Information Gain, (ii) Gini Impurity Index, (iii) Gain-Ratio, (iv) Information Gain weighted by the cost of the attribute (as defined in Tom Mitchell's Text-Book).
    - **b. Maximum Depth of the Decision Tree (*Max_Depth*)**: Any user-defined positive integer value.
    - **c. Maximum impurity (inhomogeneity) at any node to stop further branching (*Max_Impurity*):** Any user-defined positive real number within [0, 1].
    - **d. Training:Validation:Testing dataset split ratio (defined by *Train_Seg*:*Val_Seg*:(100 – *Train_Seg – Val_Seg*)):** An user-defined positive integer value within [0, 100]. Dataset partition should be made randomly with both option of "with replacement" and "without replacement".
    - **e. Noise-percentage for evaluating robustness of the learned model (*percentNoise*):** This user-defined parameter within [0, 100] defines what percentage of total attribute values in the given training dataset can be randomly changed (within the possible values for any attribute) before starting the training process.
3. **Perform experiment with Standard Set-up (Settings-0)**: [*Impurity_Metric* = Entropy-based Information Gain], [*Max_Depth* = infinity], [*Max_Impurity* = 0], [*Train_Seg* = 80, *Val_Seg* = 10], [*percentNoise* = 0].
    - **a.** Plot the gradual reduction of total impurity of the training dataset after branching at every new node of the decision tree.
    - **b.** Display the final Decision Tree as Rule-Set for every branch of the tree with the following representation [(Attribute1 = Value1) AND (Attribute2 = Value2) AND…] OR [….].
    - **c.** Find out the final accuracy values for the training, validation and testing datasets with the learned model.
    - **d.** If every node (starting from the root) is gradually considered as the leaf (classification) node of the decision tree – based on majority voting for the examples in the data subset attached to that node – then show how the accuracy of the training, validation and testing datasets changes as the nodes are sequentially considered for branching. Discuss on the resulting accuracy curves.
4. **Evaluate effects of different Hyper-parameters**: Repeat Step 3 for following settings:
    - **a.** Settings-1: Change Setting-0 as follows: Train_Seg = 60, Val_Seg = 20.
    - **b.** Settings-2: Change Setting-0 as follows: percentNoise = 10
    - **c.** Setting-3: Change Setting-1 as follows: percentNoise = 10
    - **d.** Setting-4: Change Setting-0 as follows: Max_Depth = 3, Max_Impurity = 0.25
    - **e.** Setting-5: Change Setting-0 as follows: Impurity_Metric = Gini impurity index

5. **Develop a generic Random Forest (RF) Classifier**: Utilize the Decision Trees (DT) developed under Step 3, make multiple copies of the DT model, a use the following RF specific user-defined hyper-parameters for developing the RF classifier:
    a. **Number of DT model weak learners (numTrees)**: It takes positive scalar integer numbers.
    b. **Maximum number of features considered per DT model – size of Feature-Bag (featureBagSize)**: It takes one positive scaler integer number between [1, numAttributes = |Attributes|]. For individual DT models, the random selection of attributes / features would be without-replacement. For across DT models, the random selection of attributes / features would be with-replacement. That means, same feature would repeat in individual DT models, but can be reused by multiple DT models.
    c. **Percentage of examples considered for training of individual DT model – size of the Example-Bag (dataBagSize):** It takes one positive scalar integer number between [1, 100]. Using the training-segment of the data (from Step 2d), randomly select (dataBagSize*train_Seg*totalNumData) examples for individual training datasets for individual DT models, without-replacement for individual models, and with-replacement across different DT models.
    d.
6. Train the RF classifier for given dataset: Find the following measures of the RF classifier.
    a. Average accuracy for Out-of-Bag data examples
    b. Accuracy of the RF classifier for Testing dataset and Validation dataset as per Step 2d.
    c. Find the significance of the attributes / features as per the below metrices:
        i. How many times an attribute is appearing at the Root node of the individual DT models.
        ii. Weighted significance values of the attributes / features as per the below metric:

            wtFeatureSignificance = summation_accross_DTmodels ((featureBagSize – depthOfFeature – 1) / featureBagSize)

        iii.
    d.
7. Compare and discuss the accuracy results for DT models in Step 3-4 and RF model in Step 6.

ID3(*Examples*, *Target_attribute*, *Attributes*)

Examples are the training examples. Target_attribute is the attribute whose value is to be predicted by the tree. Attributes is a list of other attributes that may be tested by the learned decision tree. Returns a decision tree that correctly classifies the given Examples.

- Create a *Root* node for the tree
- If all *Examples* are positive, Return the single-node tree *Root*, with label = +
- If all *Examples* are negative, Return the single-node tree *Root*, with label = −
- If *Attributes* is empty, Return the single-node tree *Root*, with label = most common value of *Target_attribute* in *Examples*
- Otherwise Begin
  - $A \leftarrow$ the attribute from *Attributes* that best* classifies *Examples*
  - The decision attribute for *Root* $\leftarrow A$
  - For each possible value, v_i, of A,
    - Add a new tree branch below *Root*, corresponding to the test $A = v_i$
    - Let $Examples_{v_i}$ be the subset of *Examples* that have value v_i for A
    - If $Examples_{v_i}$ is empty
      - Then below this new branch add a leaf node with label = most common value of *Target_attribute* in *Examples*
      - Else below this new branch add the subtree
        ID3($Examples_{v_i}$, *Target_attribute*, *Attributes* − {A}))
- End
- Return *Root*